

PERSONAL FINANCE ADVISOR BOT

Project Report

An AI-powered web application for personal budgeting, expense analysis and financial guidance

Submitted By	Om Moudekar
Project Type	AI / Web Application
Backend	Python Flask
AI Service	Google Gemini API

2026

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Proposed Solution
5. Project Scenarios
6. System Workflow and Architecture
7. Technology Stack
8. Database Design
9. Financial Calculations and AI Logic
10. Frontend and Dashboard
11. Testing and Deployment
12. Screenshots, GitHub and Demo
13. Conclusion and Future Enhancements

1. Project Overview

The **Personal Finance Advisor Bot** is an AI-assisted web application designed to help users understand their income and expenses and make better day-to-day budgeting decisions. The application combines a simple web interface with a Python Flask backend and the Google Gemini API. Users enter their income, spending categories, savings amount and a financial goal. The system calculates the financial summary and generates structured, personalized advice.

The project focuses on practical personal finance rather than complex investment management. It is designed to present understandable recommendations such as expense observations, budgeting suggestions, saving ideas and emergency-fund guidance. The application also validates important input values and keeps the AI response in a structured JSON format so that the frontend can display the result consistently.

Key Features

- Income and expense input through a clean web interface.
- Category-wise spending analysis for rent, food, transport, dining, entertainment, utilities and other expenses.
- Calculation of total expenses, remaining amount and planned savings.
- Goal-oriented advice, including emergency-fund planning.
- AI-generated financial observations and practical saving suggestions.
- Structured Gemini response in JSON for reliable frontend rendering.
- Input validation and controlled API interaction.

2. Problem Statement

Many students and young earners receive income but do not maintain a clear picture of where their money is being spent. Manual calculation of monthly expenses can be repetitive, and generic financial advice may not consider an individual's actual income, spending pattern and immediate goal. A lightweight system is therefore needed to combine basic financial calculations with AI-assisted explanation.

Problem: Users need a simple and accessible tool that converts raw income and expense information into a clear financial summary and understandable, goal-oriented suggestions.

3. Objectives

- Develop a user-friendly web application for entering personal income and expenses.
- Automatically calculate total expenses and the amount remaining after expenses.
- Compare planned savings with the remaining available amount.
- Provide AI-generated observations based on actual user-entered values.
- Support a selected financial goal such as building an emergency fund.
- Avoid unsupported investment guarantees and speculative financial claims.
- Create a lightweight architecture that can be run locally and exposed for demonstration using ngrok.

4. Proposed Solution

The proposed solution uses a three-part flow: a browser-based frontend collects financial information, a Flask backend validates the input and performs deterministic calculations, and the Gemini API generates contextual financial guidance. The backend constructs a controlled prompt containing the user's actual financial values and requests a JSON-only response. This separates numerical calculations from AI-generated explanation.

Solution Flow

- User enters monthly income and category-wise expenses.
- Frontend sends the data to the Flask **/analyse** endpoint.
- Backend validates the request and rejects invalid or unreasonable income values.
- Backend calculates total expenses and remaining money.
- A structured prompt is created for Gemini using the user's actual values and selected goal.
- Gemini returns structured advice in JSON format.
- Frontend displays the financial summary, analysis and suggestions.

5. Project Scenarios

Scenario	User Action	System Response
Monthly budgeting	Enters income and regular expenses	Calculates total spending and remaining amount
Expense analysis	Provides category-wise spending	Highlights spending pattern and areas to review
Savings planning	Adds planned savings	Compares planned savings with remaining money
Emergency fund	Selects emergency-fund goal	Generates goal-oriented saving suggestions
Invalid input	Submits invalid/non-positive income	Returns validation error instead of calling AI

6. System Workflow and Architecture

Architecture

Layer	Component	Responsibility
Presentation	HTML + CSS + JavaScript	Collect input and display financial results
Application	Flask / Python	Validation, calculations, prompt creation and API route
AI	Google Gemini API	Generate contextual financial guidance in structured JSON

Layer	Component	Responsibility
Configuration	.env / environment variables	Store sensitive API configuration outside source code

Workflow

- 1. Browser → User enters financial data.
- 2. JavaScript → Sends JSON request to **/analyse**.
- 3. Flask → Validates income and expense data.
- 4. Python → Calculates deterministic financial totals.
- 5. Flask → Builds a controlled AI prompt.
- 6. Gemini → Returns JSON-formatted financial advice.
- 7. Flask → Sends the result to the browser.
- 8. Dashboard → Displays summary, observations and suggestions.

7. Technology Stack

Technology	Purpose
Python	Backend programming and financial calculations
Flask	Web server and REST-style analysis endpoint
HTML5	Application structure and form elements
CSS3	Responsive visual design and dashboard styling
JavaScript	Frontend interaction and API communication
Google Gemini API	AI-generated personalized financial guidance
JSON	Data exchange between frontend, backend and AI response
ngrok	Temporary public tunnel for local demonstration
Git / GitHub	Version control and source-code hosting

8. Database Design

The current lightweight version does not require a persistent database. Financial input is processed for the current analysis request and is not dependent on a database server. This keeps the prototype simple, portable and easy to demonstrate.

Future Database Design

Entity	Important Fields	Purpose
User	user_id, name, email, created_at	Optional account management
FinancialProfile	profile_id, user_id, income, savings_target	Store financial profile information
Expense	expense_id, user_id, category, amount, month	Store historical expenses

Entity	Important Fields	Purpose
Goal	goal_id, user_id, goal_type, target_amount, status	Track financial goals
Analysis	analysis_id, user_id, totals, advice, created_at	Maintain analysis history

9. Financial Calculations and AI Logic

Financial Calculations

- **Total Expenses = Rent + Food + Transport + Dining + Entertainment + Utilities + Other Expenses**
- **Remaining Amount = Monthly Income – Total Expenses**
- **Savings Comparison = Planned Savings compared with Remaining Amount**

These numerical values are calculated by the application so that basic arithmetic does not depend on the AI model. The AI layer is then used to explain the situation and provide contextual suggestions.

AI Logic

- The backend builds a prompt containing the actual user-entered income, expense values and selected goal.
- The prompt instructs the model to return JSON only, making the response easier for the frontend to parse.
- The AI is instructed to use INR values and the user's actual data.
- The system asks for practical advice and avoids investment guarantees or speculative claims.
- Temporary API failures such as rate limits or service unavailability are handled with limited retries.

10. Frontend and Dashboard

The frontend is designed around a simple financial-advisor experience. The interface presents the application as **Finance Advisor — Smart money decisions, made simple**. Users enter income and category-wise expenses, select a goal and submit the information for analysis.

- Input section for income and expense categories.
- Goal selection including emergency-fund planning.
- Summary of total expenses and remaining amount.
- AI-generated analysis and recommendations.
- Clear presentation suitable for students and first-time budget planners.
- Responsive HTML/CSS/JavaScript implementation without requiring a heavy frontend framework.

11. Testing and Deployment

Testing

Test Case	Input / Condition	Expected Result
Valid analysis	Positive income and valid expense values	Financial summary and AI advice are returned

Test Case	Input / Condition	Expected Result
Zero income	Income $\leq$ 0	Validation error; analysis is not performed
Very large income	Income above configured limit	Validation error
Missing/invalid data	Malformed request values	Request is rejected safely
AI temporary failure	503/rate-limit condition	Limited retry mechanism attempts recovery
Frontend endpoint	Submit analysis form	Request reaches Flask /analyse endpoint

Deployment

The application can be run locally using the Flask development server. For demonstrations, ngrok can create a public HTTPS tunnel to the local application. The project source is maintained using Git and GitHub. API secrets should remain in environment variables and should not be committed to the public repository.

12. Screenshots, GitHub and Demo

The final submission should include screenshots of the main dashboard, the input form, the generated financial analysis and the deployed demonstration. Screenshots provide visual evidence of the working application.

Suggested Screenshot Set

- Screenshot 1 — Main Personal Finance Advisor dashboard.
- Screenshot 2 — Income and expense input screen.
- Screenshot 3 — Generated financial summary and AI recommendations.
- Screenshot 4 — Running Flask application / browser demonstration.
- Screenshot 5 — GitHub repository showing project files and README.
- Screenshot 6 — ngrok public demo page or terminal showing the active tunnel.

GitHub Repository

Project repository: **omwarade17/personal-ai-finance-advisor**. The repository contains the Flask backend, frontend assets, requirements file, README and environment-variable template used for setup.

Demo URL: The ngrok URL is temporary and may change whenever a new tunnel session is started. The final report can include the active demonstration URL captured during the project presentation.

13. Conclusion and Future Enhancements

The Personal Finance Advisor Bot demonstrates how a lightweight web application can combine deterministic financial calculations with generative AI to provide understandable, personalized budgeting guidance. The project provides a practical workflow from data entry to calculation, AI analysis and dashboard presentation while keeping the implementation relatively simple and suitable for

demonstration.

Future Enhancements

- Add user authentication and secure personal profiles.
- Introduce a persistent database for monthly financial history.
- Add charts for category-wise and month-wise spending trends.
- Support recurring expenses and automatic monthly budgets.
- Add goal tracking for emergency funds and other savings goals.
- Provide downloadable monthly financial reports.
- Add stronger privacy controls and encryption for stored financial data.
- Deploy the backend on a production hosting platform instead of relying on a temporary ngrok tunnel.
- Add optional notifications for budget limits and savings milestones.

Project Prepared By: Om Moudekar

Personal Finance Advisor Bot — AI-assisted budgeting and financial guidance